



Aerofit Treadmill Business Case Study

1. Import Libraries

```
In [ ]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

2. Load Dataset

```
In [ ]: df = pd.read_csv('aerofit_treadmill.csv')
df.head()
```

```
Out[ ]:
```

	Product	Age	Gender	Education	MaritalStatus	Usage	Fitness	Income	Miles
0	KP281	18	Male	14	Single	3	4	29562	1
1	KP281	19	Male	15	Single	2	3	31836	
2	KP281	19	Female	14	Partnered	4	3	30699	
3	KP281	19	Male	12	Single	3	3	32973	
4	KP281	20	Male	13	Partnered	4	2	35247	

3. Dataset Overview

```
In [ ]: df.shape
df.info()
df.describe()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 180 entries, 0 to 179
Data columns (total 9 columns):
#   Column          Non-Null Count  Dtype  
---  -
0   Product         180 non-null   object  
1   Age             180 non-null   int64   
2   Gender          180 non-null   object  
3   Education       180 non-null   int64   
4   MaritalStatus   180 non-null   object  
5   Usage          180 non-null   int64   
6   Fitness         180 non-null   int64   
7   Income          180 non-null   int64   
8   Miles           180 non-null   int64   
dtypes: int64(6), object(3)
memory usage: 12.8+ KB
```

```
Out[ ]:
```

	Age	Education	Usage	Fitness	Income	Mile
count	180.000000	180.000000	180.000000	180.000000	180.000000	180.00000
mean	28.788889	15.572222	3.455556	3.311111	53719.577778	103.19444
std	6.943498	1.617055	1.084797	0.958869	16506.684226	51.86360
min	18.000000	12.000000	2.000000	1.000000	29562.000000	21.00000
25%	24.000000	14.000000	3.000000	3.000000	44058.750000	66.00000
50%	26.000000	16.000000	3.000000	3.000000	50596.500000	94.00000
75%	33.000000	16.000000	4.000000	4.000000	58668.000000	114.75000
max	50.000000	21.000000	7.000000	5.000000	104581.000000	360.00000

4. Missing Values

```
In [ ]: df.isnull().sum()
```

```
Out[ ]:
```

Product	0
Age	0
Gender	0
Education	0
MaritalStatus	0
Usage	0
Fitness	0
Income	0
Miles	0

dtype: int64

5. Convert Categorical Columns

```
In [ ]: df['Product'] = df['Product'].astype('category')
df['Gender'] = df['Gender'].astype('category')
df['MaritalStatus'] = df['MaritalStatus'].astype('category')
```

6. Value Counts (Non-Graphical Analysis)

```
In [ ]: df['Product'].value_counts()
```

```
Out[ ]:
```

count	
Product	
KP281	80
KP481	60
KP781	40

dtype: int64

```
In [ ]: df['Gender'].value_counts()
```

```
Out[ ]:
```

count	
Gender	
Male	104
Female	76

dtype: int64

```
In [ ]: df['MaritalStatus'].value_counts()
```

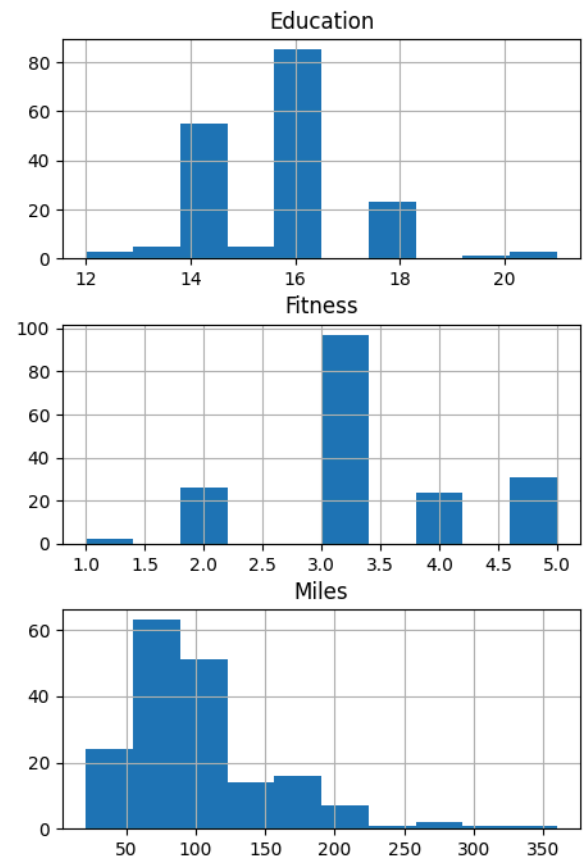
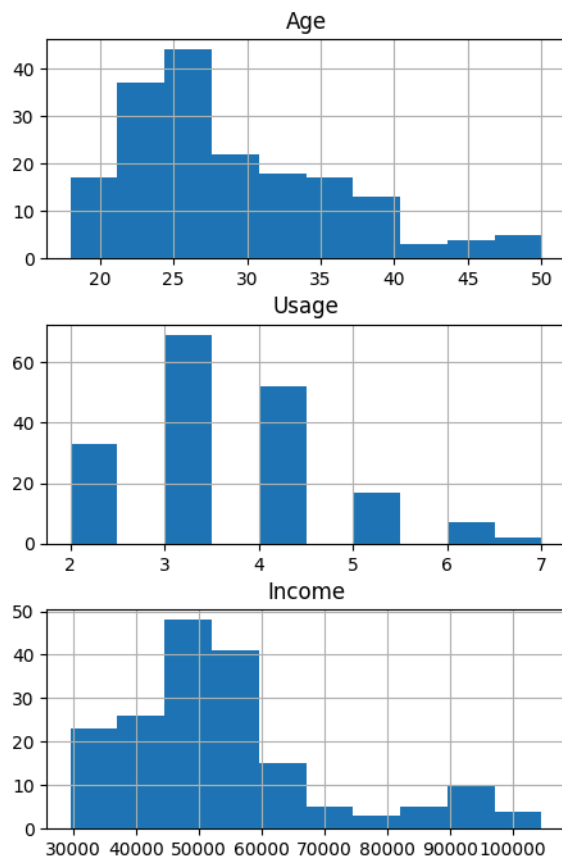
```
Out[ ]:
```

count	
MaritalStatus	
Partnered	107
Single	73

dtype: int64

7. Univariate Analysis

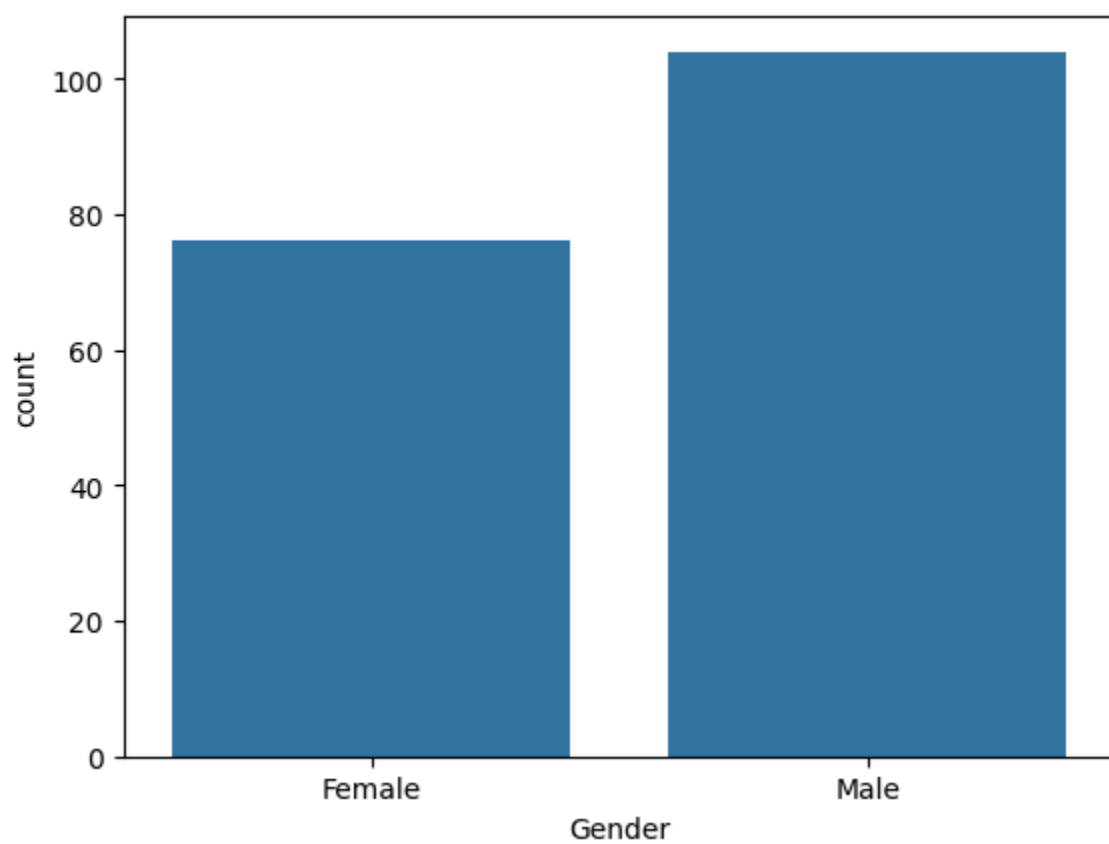
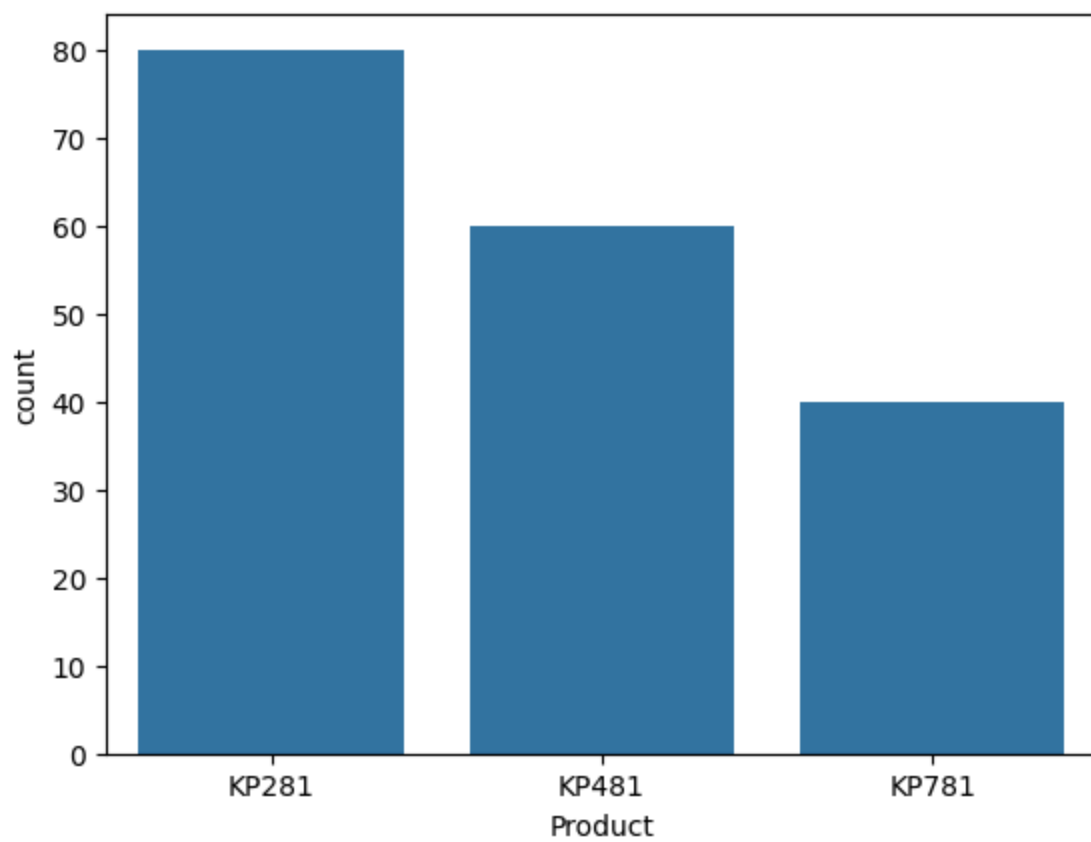
```
In [ ]: df.hist(figsize=(12,8))  
plt.show()
```

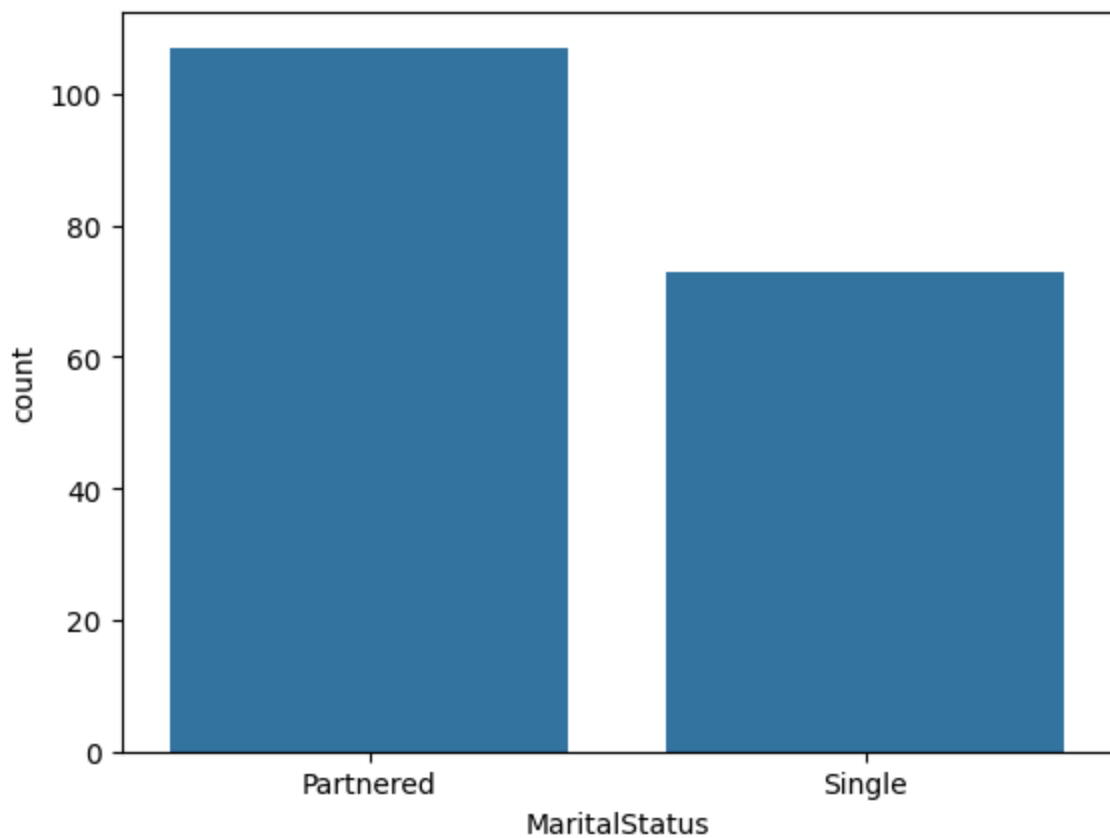


```
In [ ]: sns.countplot(x='Product', data=df)
plt.show()

sns.countplot(x='Gender', data=df)
plt.show()

sns.countplot(x='MaritalStatus', data=df)
plt.show()
```



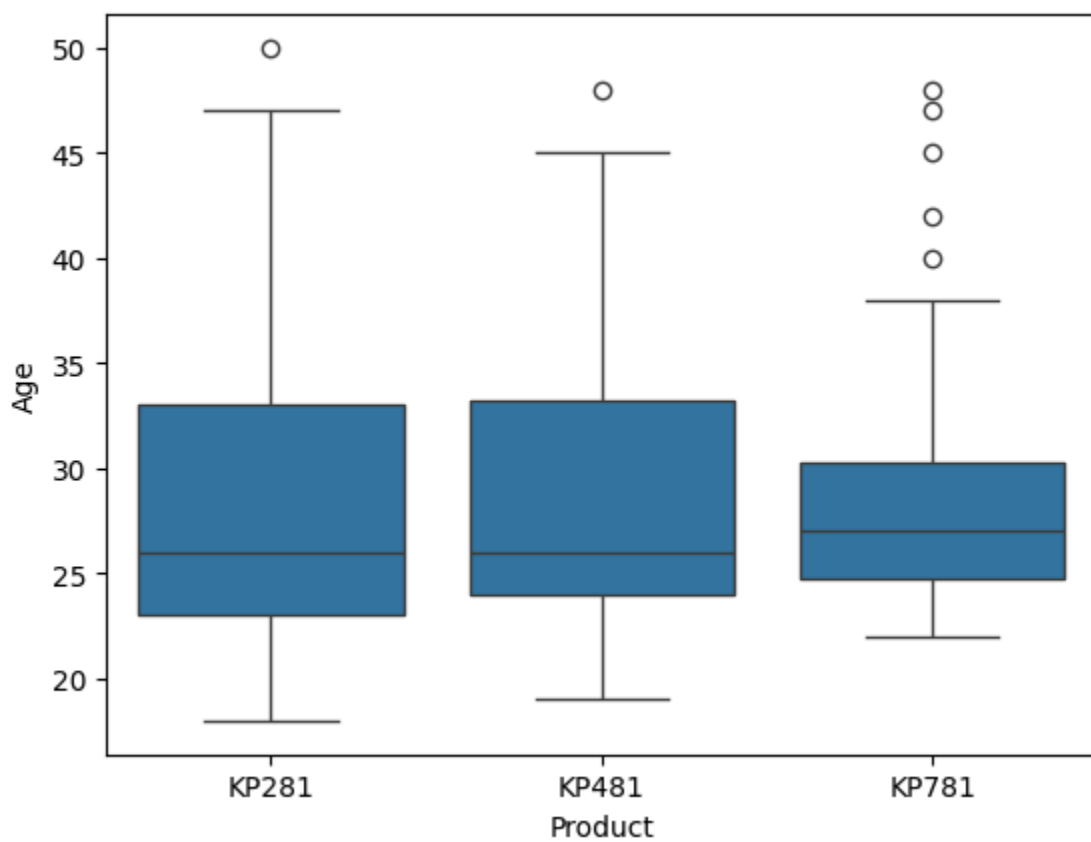
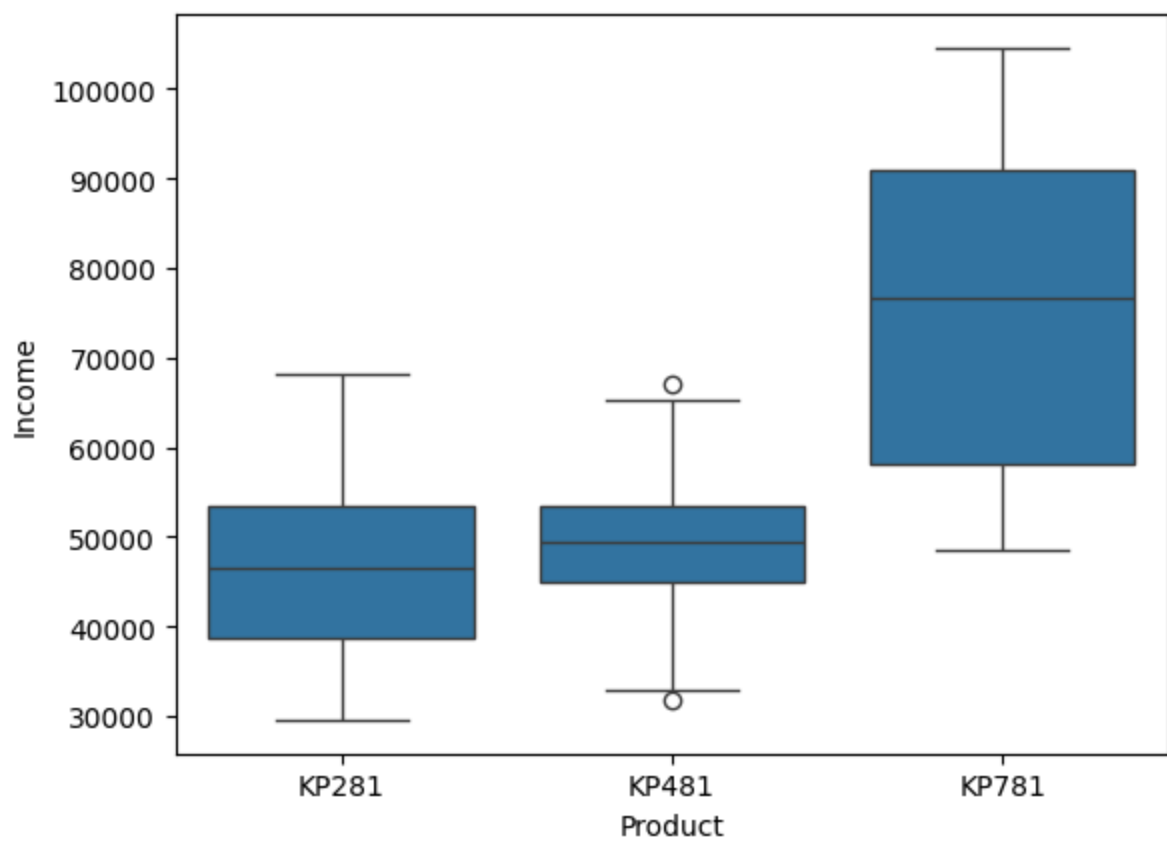


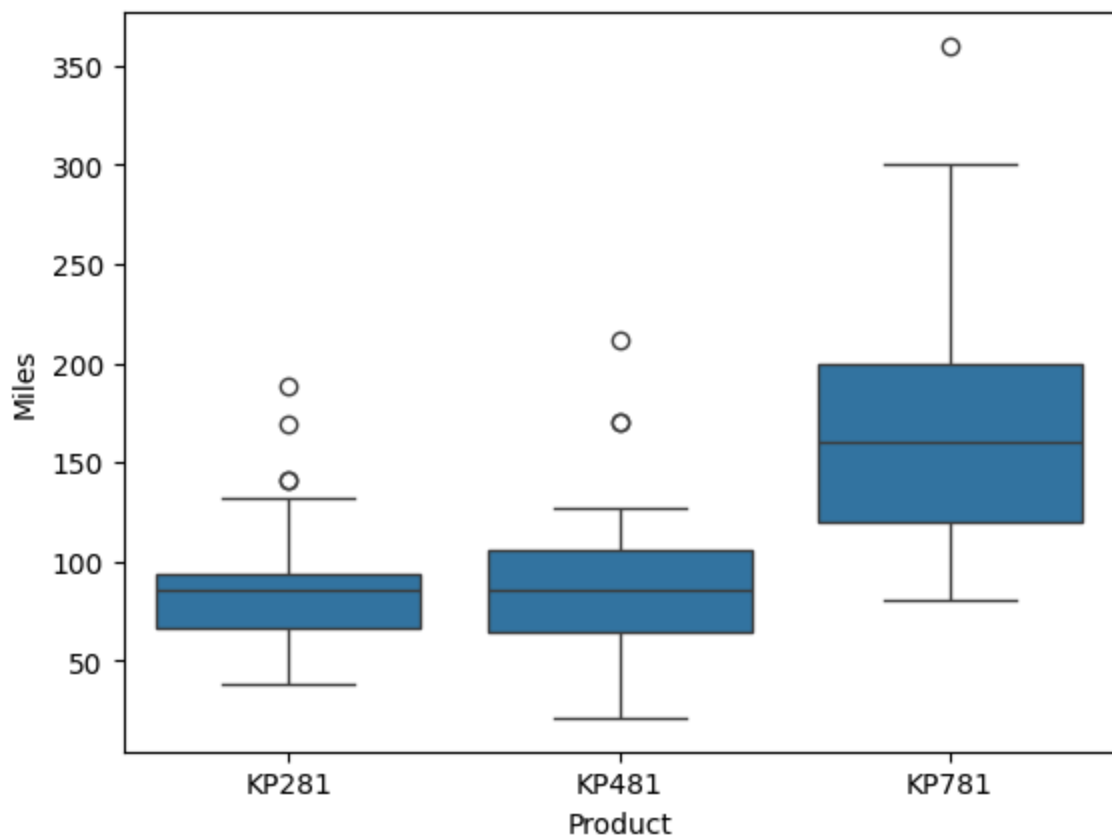
8. Bivariate Analysis (Boxplots)

```
In [ ]: sns.boxplot(x='Product', y='Income', data=df)
plt.show()

sns.boxplot(x='Product', y='Age', data=df)
plt.show()

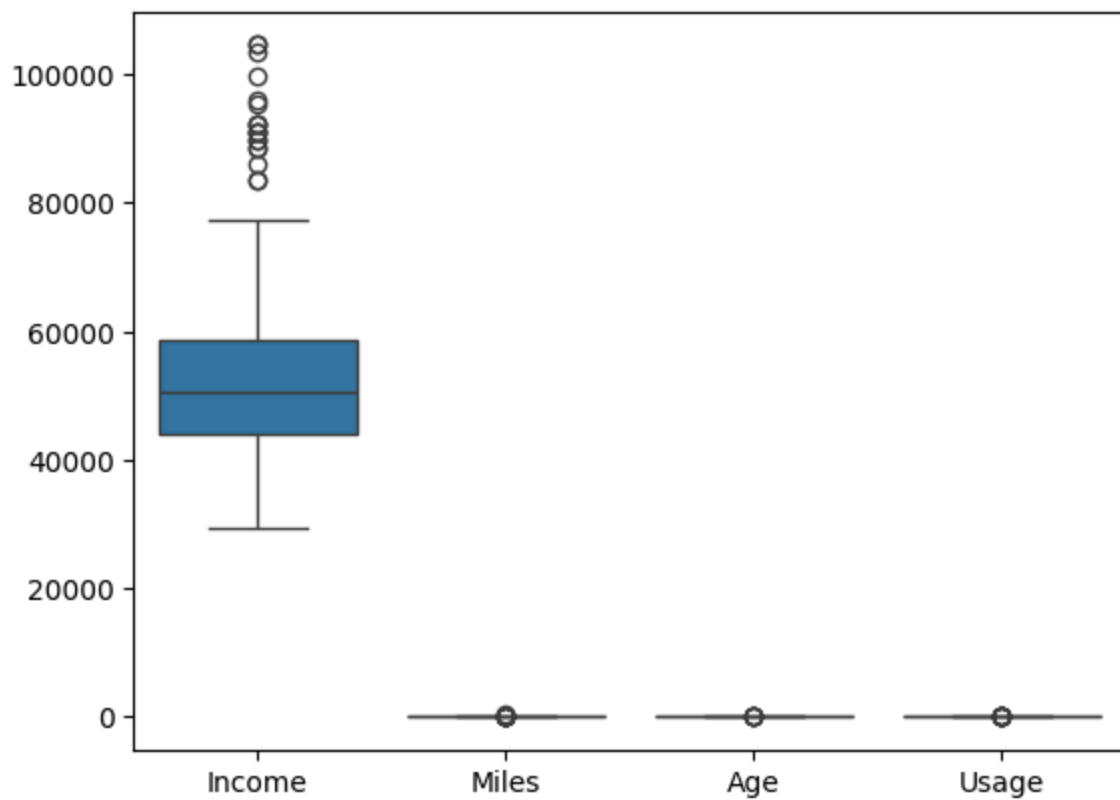
sns.boxplot(x='Product', y='Miles', data=df)
plt.show()
```





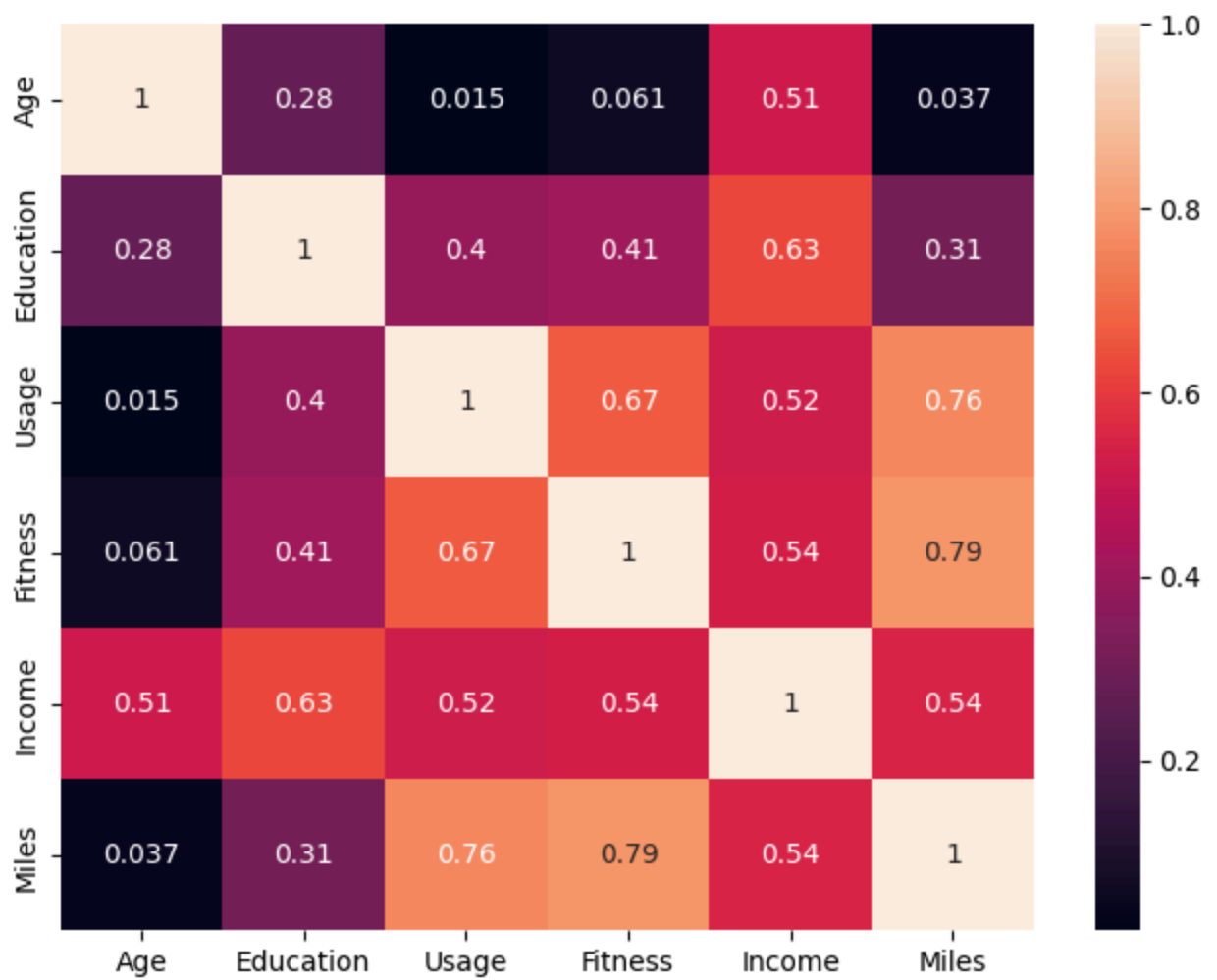
9. Outlier Detection

```
In [ ]: sns.boxplot(data=df[['Income', 'Miles', 'Age', 'Usage']])  
plt.show()
```

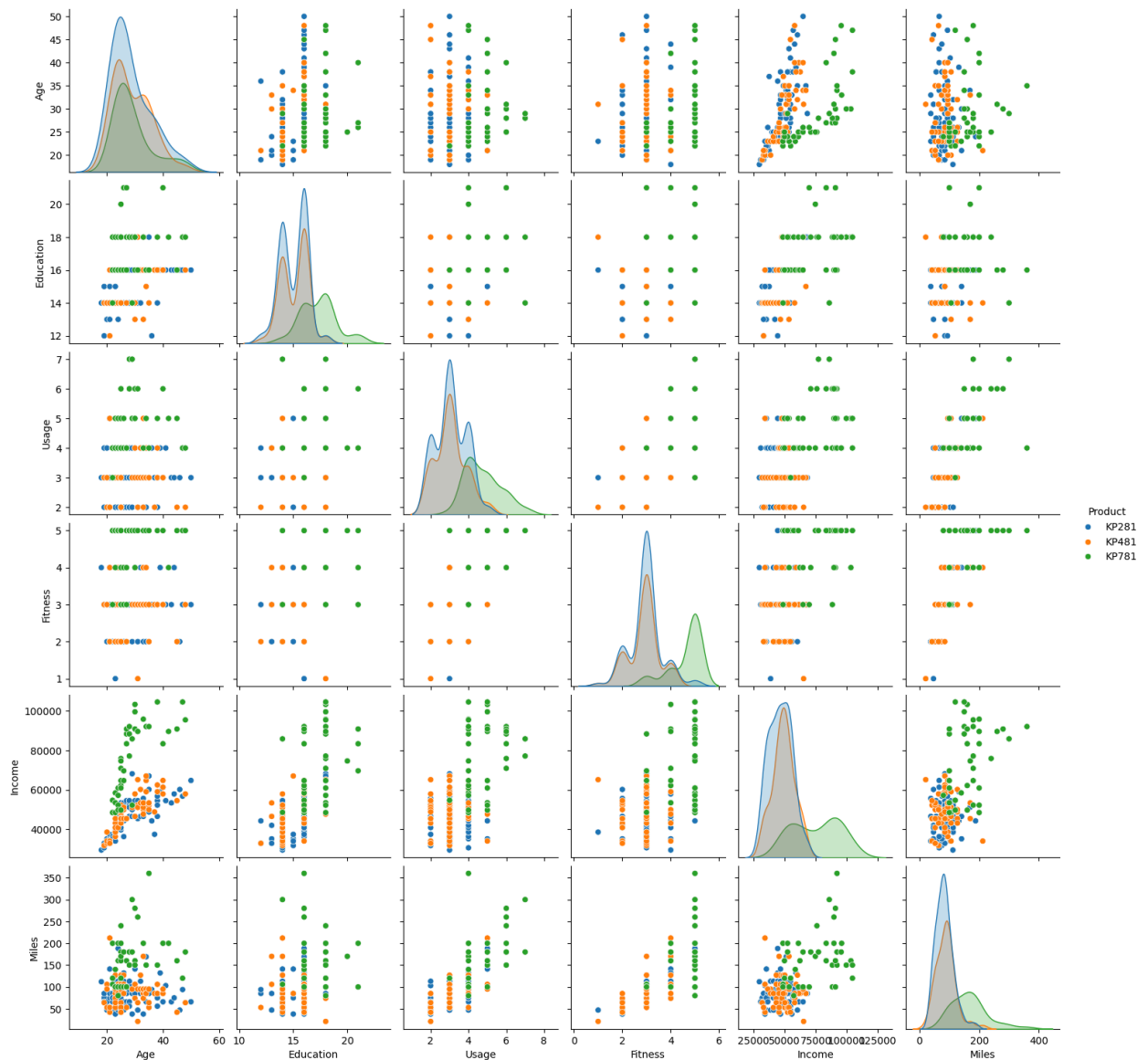
10. Correlation Heatmap

```
In [ ]: plt.figure(figsize=(8,6))
sns.heatmap(df.corr(numeric_only=True), annot=True)
plt.show()
```



11. Pairplot

```
In [ ]: sns.pairplot(df, hue='Product')  
plt.show()
```



12. Marginal Probability of Product Purchase

```
In [ ]: product_prob = df['Product'].value_counts(normalize=True)
product_prob
```

```
Out[ ]:          proportion
```

Product

KP281 0.444444

KP481 0.333333

KP781 0.222222

dtype: float64

13. Contingency Table (Product vs Gender)

```
In [ ]: pd.crosstab(df['Product'], df['Gender'])
```

```
Out[ ]:  Gender  Female  Male
        Product
        KP281      40     40
        KP481      29     31
        KP781       7     33
```

```
In [ ]: pd.crosstab(df['Product'], df['Gender'], normalize=True)
```

```
Out[ ]:  Gender  Female  Male
        Product
        KP281  0.222222  0.222222
        KP481  0.161111  0.172222
        KP781  0.038889  0.183333
```

```
In [ ]: pd.crosstab(df['Product'], df['Gender'], normalize='columns')
```

```
Out[ ]:  Gender  Female  Male
        Product
        KP281  0.526316  0.384615
        KP481  0.381579  0.298077
        KP781  0.092105  0.317308
```

14. Contingency Table (Product vs Marital Status)

```
In [ ]: pd.crosstab(df['Product'], df['MaritalStatus'], normalize='columns')
```

Out[]:	MaritalStatus	Partnered	Single
Product			
	KP281	0.448598	0.438356
	KP481	0.336449	0.328767
	KP781	0.214953	0.232877

15. Business Insights

- Entry level treadmill KP281 is usually purchased by lower income or beginner runners.
- KP481 customers tend to have moderate income and moderate fitness.
- KP781 buyers typically have higher income and higher fitness scores.
- Income and fitness level influence treadmill choice.
- Customers running 15+ miles per week prefer advanced treadmills.
- Fitness level influences premium product adoption

16. Recommendations

1. Market KP281 to beginners and budget customers.
2. Promote KP481 as a mid-range option for regular runners.
3. Target KP781 toward high-income and high-fitness customers.
4. Provide EMI options for premium treadmills to increase adoption.
5. Use fitness level and income data for personalized product recommendations.